

PYTHON

ORM SQLAlchemy

PRÁCTICA 1-M6



Práctica: Sistema de Gestión de una Agencia de Viajes con ORM SQLAlchemy

1. Introducción

En esta práctica se desarrollará un sistema de gestión para una agencia de viajes utilizando SQLAlchemy y SQLite dentro de un entorno de desarrollo en PyCharm. Se implementará un ORM (Object Relational Mapping) para gestionar la base de datos sin necesidad de escribir consultas SQL directamente.

Se trabajará con un sistema modular que dividirá la aplicación en varios archivos para mejorar la organización del código y facilitar su mantenimiento.

2. Objetivos

- Configurar un proyecto en PyCharm con SQLAlchemy y SQLite.
- Definir clases y relaciones en `models.py` usando SQLAlchemy.
- Implementar una configuración centralizada de la base de datos en `db.py`, almacenando la base de datos dentro de `database/`.
- Utilizar `main.py` para manejar la lógica principal de la aplicación.
- Aplicar operaciones CRUD en la base de datos.
- Consultar y analizar la información almacenada en la base de datos.



3. Estructura del Proyecto

📁 AgenciaViajes (*Carpeta del proyecto en PyCharm*)

- | — 📁 database (*Carpeta donde se almacenará la base de datos*)
 - | | — 📄 agencia_viajes.db (*Base de datos SQLite, se generará automáticamente*)
- | — 📄 db.py (*Configuración de la base de datos y sesión*)
- | — 📄 models.py (*Definición de clases y relaciones en la base de datos*)
- | — 📄 main.py (*Lógica principal de la aplicación*)

Cada archivo cumple una función específica:

- models.py → Define las clases de SQLAlchemy y las relaciones en la base de datos.
- db.py (*en la raíz*) → Configura la conexión a la base de datos y la sesión de SQLAlchemy.
- main.py → Se encarga de la lógica principal del sistema, como insertar, consultar y modificar datos.

4. Instalación y Configuración del Entorno en PyCharm

Antes de comenzar con el desarrollo, es necesario instalar las dependencias y configurar el entorno:

1. Abrir PyCharm y crear un nuevo proyecto llamado AgenciaViajes.
2. Crear un entorno virtual (Virtual Environment - venv) dentro del proyecto.
3. Instalar SQLAlchemy ejecutando el siguiente comando en la terminal de PyCharm:

pip install sqlalchemy

4. Crear las carpetas y archivos necesarios siguiendo la estructura anterior.
-



5. Definición del Modelo de Datos en models.py

Las siguientes entidades se implementarán en la base de datos:

- Destino: Un destino turístico disponible para los clientes.
- GuiaTuristico: Un guía que trabaja en la agencia y puede acompañar a varios viajes.
- Cliente: Un cliente registrado en la agencia que puede reservar viajes.
- Pasaporte: Un documento de identificación que cada cliente tiene.
- Viaje: Un paquete de viaje ofrecido por la agencia.
- Reserva: Una reserva realizada por un cliente para un viaje.

Se deben definir correctamente las relaciones:

1. Relación 1:N (Uno a Muchos)
 - Un destino puede tener varias reservas.
 - Un guía puede estar asignado a múltiples viajes.
2. Relación 1:1 (Uno a Uno)
 - Cada cliente tiene un único pasaporte.
3. Relación N:N (Muchos a Muchos)
 - Un cliente puede reservar múltiples viajes, y un viaje puede tener múltiples clientes.

6. Configuración de la Base de Datos en db.py

Para manejar la base de datos de manera eficiente, la configuración se centralizará en db.py.

Pasos a seguir en db.py

1. Importar SQLAlchemy y crear el motor de la base de datos SQLite dentro de la carpeta database/.
2. Definir la sesión de SQLAlchemy.
3. Crear la base de datos si no existe dentro de database/agencia_viajes.db.



Ubicación de la base de datos

- agencia_viajes.db se almacenará dentro de la carpeta database/, pero la configuración seguirá estando en db.py en la raíz del proyecto.

7. Inserción de Datos y Operaciones CRUD en main.py

Una vez configurados los modelos y la base de datos, se deben realizar las siguientes operaciones desde main.py:

1. Insertar destinos turísticos, guías, clientes y viajes.
2. Realizar reservas de viajes.
3. Consultar la información de la base de datos (viajes disponibles, clientes con reservas, guías asignados a viajes, etc.).
4. Modificar y eliminar datos de clientes y reservas.

8. Consultas en la Base de Datos

Se deben realizar consultas para obtener información sobre los datos almacenados:

1. Obtener la lista de destinos turísticos disponibles.
2. Consultar los viajes programados y sus guías asignados.
3. Obtener los clientes que han reservado un viaje a un destino específico.
4. Buscar clientes por número de pasaporte.
5. Obtener la cantidad de reservas realizadas para cada destino.

9. Retos Adicionales

Una vez completada la implementación básica, se proponen los siguientes desafíos adicionales para mejorar el sistema:

1. Añadir una tabla de pagos, donde cada reserva tenga un estado de pago (pendiente, pagado, cancelado).



2. Crear un menú interactivo en la terminal para permitir a los usuarios realizar consultas y registrar reservas de manera dinámica.
 3. Implementar un sistema de disponibilidad de cupos en los viajes, evitando que se pueda reservar un viaje si ya no hay cupos disponibles.
 4. Generar un informe de ventas con los viajes más reservados.
 5. Exportar la lista de clientes y reservas a un archivo CSV.
-

10. Evaluación de la Práctica

Para que la práctica sea considerada completa, se evaluará lo siguiente:

- Implementación correcta de las clases y relaciones en `models.py`.
- Configuración adecuada de la base de datos en `db.py`.
- Uso de operaciones CRUD en `main.py`.
- Aplicación de consultas avanzadas en SQLAlchemy.
- Correcto manejo de relaciones 1:N, 1:1 y N:N.
- Modularidad y organización del código en distintos archivos.

Se recomienda probar el sistema con diferentes datos y verificar que todas las operaciones funcionan correctamente.

11. Conclusión

Con esta práctica, se aplicará SQLAlchemy en un escenario realista para gestionar información de una agencia de viajes. Se trabajará con datos estructurados en una base de datos relacional y se aprenderá a utilizar un ORM para manipular la información sin necesidad de escribir SQL directamente.

Este enfoque modular facilita la escalabilidad del proyecto y permite integrar más funcionalidades en el futuro.



12. ENTREGA

⚠ IMPORTANTE: ENTREGA DE LA PRÁCTICA ⚠

Para la correcta entrega de esta práctica, se debe seguir estrictamente las siguientes indicaciones:

El documento final debe ser un archivo PDF con el siguiente nombre de archivo obligatorio:

M6_01_nombre_apellido1_apellido2.pdf

(Sustituir 'nombre', 'apellido1' y 'apellido2' por vuestros datos personales)

El PDF debe contener:

- Captura de pantalla de vuestro entorno de trabajo en PyCharm, donde se vea claramente:
 - A la izquierda: La estructura de ficheros del proyecto.
 - A la derecha: El código principal del archivo main.py.
- Listado de consultas SQL realizadas, en formato texto, con la siguiente estructura:

CONSULTA 1:

```
SELECT * FROM clientes;
```

RESULTADO CONSULTA 1:

id	nombre	email	
----	-----	-----	
1	Juan Pérez		juan@example.com
2	María Gómez		maria@example.com

ETC.



- Código fuente completo de la práctica, en las páginas siguientes, copiando y pegando en orden:
 - Código de main.py
 - Código de models.py
 - Código de db.py

 NO SE ACEPTARÁN ENTREGAS QUE NO SIGAN ESTOS REQUISITOS.

 Revisad que el PDF contenga toda la información solicitada antes de entregarlo.